

Exercises on MapReduce

Exercise 1 Design an MR algorithm which solves (exactly) the Maximum pairwise distance problem for a set S of N points from a metric space (M, d) . The algorithm must run in $R = O(1)$ rounds and require local space $M_L = O(\sqrt{N})$ and aggregate space $M_A = O(N^2)$.

Solution. The input-output specification of the problem is the following one:

- **Input:** Set S of N points represented by pairs (i, x_i) , for $0 \leq i < N$.
- **Output:** $(0, d_{\max} = \max_{0 \leq i, j < N} d(x_i, x_j))$.

The idea is first to compute the distances between all possible pairs x_i, x_j with $0 \leq i \leq j < N$ (this is done in the first 2 rounds), and then to compute the maximum among them in several rounds, processing distances in groups of size $\sqrt{N}$. The details of the algorithm are below.

Round 1

- *Map phase:* map each pair (i, x_i) into the following $\sqrt{N}$ intermediate pairs: $((i, j), x_i)$, with $0 \leq j < \sqrt{N}$.
- *Reduce phase:* empty

Round 2

- *Map phase:* map each pair $((i, j), x_i)$ into the following $\sqrt{N}$ intermediate pairs: $((i, j * \sqrt{N} + j'), x_i)$, with $0 \leq j' < \sqrt{N}$.
- *Reduce phase:* empty

Observation: At this point, for every x_i we have N pairs $((i, j), x_i)$, for $0 \leq j < N$.

Round 3

- *Map phase:* map each pair $((i, j), x_i)$ into the pair $((\min\{i, j\}, \max\{i, j\}), x_i)$.
- *Reduce phase:* for every $0 \leq i \leq j < N$ separately, gather the two intermediate pairs with key (i, j) into pair $((i, j), L_{i,j})$, where $L_{i,j}$ is a list containing x_i and x_j , and return pair $(i * N + j, d(x_i, x_j))$.

Observation: At this point we have all pairwise distances among points, represented in $N(N+1)/2 < N^2$ pairs with distinct integer keys in $[0, N^2)$. (Obs. Rounds 2 and 3 can be easily merged into one round, but we kept them separate for clarity.)

Round 4

- *Map phase*: map each pair (k, d) into the intermediate pair $(k' = k \bmod N^{3/2}, d)$.
- *Reduce phase*: for each $k' \in [0, N^{3/2})$ separately, gather the intermediate pairs with key k' into pair $(k', L_{k'})$, where $L_{k'}$ is the list of values (i.e., distances) from these pairs, and return pair $(k', d' = \max\{d \in L_{k'}\})$. Note that $|L_{k'}| \leq \sqrt{N}$.

Round 5

- *Map phase*: map each pair (k, d) into the intermediate pair $(k' = k \bmod N, d)$.
- *Reduce phase*: for each $k' \in [0, N)$ separately, gather the intermediate pairs with key k' into pair $(k', L_{k'})$, where $L_{k'}$ is the list of values (i.e., distances) from these pairs, and return pair $(k', d' = \max\{d \in L_{k'}\})$. Note that $|L_{k'}| \leq \sqrt{N}$.

Round 6

- *Map phase*: map each pair (k, d) into the intermediate pair $(k' = k \bmod \sqrt{N}, d)$.
- *Reduce phase*: for each $k' \in [0, \sqrt{N})$ separately, gather the intermediate pairs with key k' into pair $(k', L_{k'})$, where $L_{k'}$ is the list of values (i.e., distances) from these pairs, and return pair $(k', d' = \max\{d \in L_{k'}\})$. Note that $|L_{k'}| \leq \sqrt{N}$.

Round 7

- *Map phase*: identity map.
- *Reduce phase*: gather the $\leq \sqrt{N}$ pairs $(0, d)$ into pair $(0, L)$, where L is the list of values from these pairs, and return $(0, \max_{d \in L} d)$ as final output.

It is easy to see that the above 7-round algorithm uses aggregate space proportional to the number of ordered pairs of points, that is, $M_A = O(N^2)$. Let us analyze the local space. Each map phase of Rounds 1 and 2 transforms a single pair into $\sqrt{N}$ intermediate pairs, thus requiring $O(\sqrt{N})$ local space, while for the map phases of the other rounds $O(1)$ local space is sufficient. Each non-empty reduce phase works separately on lists of at most $O(\sqrt{N})$ values and produces one pair out of each list, thus requiring $O(\sqrt{N})$ local space. For the above considerations it follows that $M_L = O(\sqrt{N})$. $\square$

Exercise 2 (Exercise 2.3.1.(b) of [LRU14]) *Design an $O(1)$ -round MapReduce algorithm to compute the arithmetic mean of a set S of N integers, using $O(\sqrt{N})$ local space and linear aggregate space. Assume that S is provided in input as a set of key-value pairs (i, x_i) where x_i is an arbitrary integer and i , with $0 \leq i < N$, is the key of the pair.*

Solution. The input-output specification of the problem is the following one:

- **Input:** $S = \{(i, x_i) : 0 \leq i < N\}$.

- **Output:** $(0, \text{mean}(S))$, where $\text{mean}(S) = (1/N) \sum_{i=0}^{N-1} x_i$.

While the trivial 1-round algorithm uses $\Theta(N)$ local space, the space requirements can be substantially lowered using the following simple 2-round strategy. Let us assume that the input size N is a global variable.

Round 1

- *Map phase:* map each pair (i, x_i) into the intermediate pair $(i \bmod \sqrt{N}, x_i)$.
- *Reduce phase:* for each $k \in [0, \sqrt{N})$, gather the intermediate pairs (k, x) into pair (k, L_k) , where L_k is the list of values from these pairs, and return pair $(0, \text{sum}_k = \sum_{x \in L_k} x)$.

Round 2

- *Map phase:* empty.
- *Reduce phase:* gather all pairs $(0, \text{sum}_k)$ into pair $(0, L)$, where L is the list of values of these pairs, and return pair $(0, (1/N) \sum_{\text{sum}_k \in L} \text{sum}_k)$.

It is immediate to see that the algorithm requires $M_L = \Theta(\sqrt{N})$ and aggregate space $M_A = \Theta(N)$. $\square$

Exercise 3 (Exercise 2.3.1.(d) of [LRU14]) *Design an $O(1)$ -round MapReduce algorithm to compute the number of distinct integers in a set S of N integers, using $O(\sqrt{N})$ local space and linear aggregate space. Assume that S is provided in input as a set of key-value pairs (i, x_i) where x_i is an arbitrary integer and i , with $0 \leq i < N$, is the key of the pair.*

Solution. The input-output specification of the problem is the following one:

- **Input:** $S = \{(i, x_i) : 0 \leq i < N\}$.
- **Output:** $(0, \text{count})$, where count is the number of distinct integers in S .

We present a straightforward 2-round algorithm and a more space-efficient 4-round algorithm. The 2-round algorithm does a simple removal of duplicates as follows:

Round 1

- *Map phase:* map each pair (i, x_i) into the intermediate pair (x_i, i) , that is, x_i becomes the key.
- *Reduce phase:* for each integer x separately, gather all intermediate pairs with key x into pair (x, L_x) , where L_x is the list of values of these pairs (i.e., indices i such that $x_i = x$), and return the unique pair $(0, x)$.

Observation: At this point all duplicates have been removed.

Round 2

- *Map phase*: empty.
- *Reduce phase*: gather all pairs $(0, x)$ into $(0, L)$, where L is the list of their values (i.e., the distinct integers) and return $(0, |L|)$.

The aggregate space of the algorithm is $M_A = O(N)$, since there is no replication of data but only aggregation and elimination. To analyze the local space, we define N_1 as the maximum number of occurrences of the same integer and N_2 as the number of distinct integers in the input set S . The map phase of Round 1 requires $O(1)$ local space, while the reduce phases of Rounds 1 and 2 require $\Theta(N_1)$ and $\Theta(N_2)$ local space, respectively. Hence, $M_L = \Theta(\max\{N_1, N_2\})$. Note that since $N_1 \cdot N_2 \geq N$, we have $\max\{N_1, N_2\} \geq \sqrt{N}$, however, both N_1 and N_2 can grow as large as N , asymptotically, hence M_L can be proportional to N in the worst case.

The following alternative strategy reduces the local space to the desired bound. The idea is first to remove all duplicates working on subsets of size at most $\sqrt{N}$ each (Rounds 1 and 2), and then to count the number of surviving, and at this point distinct, integers, counting at most $\sqrt{N}$ of them at once (Rounds 3 and 4)

Round 1

- *Map phase*: map each pair (i, x_i) into the intermediate pair $(i \bmod \sqrt{N}, x_i)$.
- *Reduce phase*: for each key $j \in [0, \sqrt{N})$ separately, gather all intermediate pairs (j, x) into (j, L_j) , where L_j is the list of values from these pairs, and return a pair (j, x) for each distinct $x \in L_j$ (i.e., we remove duplicates within L_j).

Observation: after Round 1, for every integer x there are at most $\sqrt{N}$ pairs (j, x) , one for each value of $j \in [0, \sqrt{N})$.

Round 2

- *Map phase*: map each pair (j, x) into the intermediate pair (x, j) .
- *Reduce phase*: for each x separately, gather all intermediate pairs (x, j) into (x, L_x) , where L_x is the set of values from these pairs, and return one pair (x, j) with j arbitrarily selected from L_x .

Observation: after Round 2, for every integer x only one pair survives, and for each value of $j \in [0, \sqrt{N})$ there are at most $\sqrt{N}$ pairs with value j .

Round 3

- *Map phase*: map each pair (x, j) into the intermediate pair (j, x) .
- *Reduce phase*: for each $j \in [0, \sqrt{N})$ separately, gather all intermediate pairs (j, x) into (j, L_j) , where L_j is the list of values from these pairs, and return pair $(0, c_j = |L_j|)$. Observe that c_j is a partial count of the distinct integers.

Round 4

- *Map phase*: empty.
- *Reduce phase*: gather all pairs $(0, c_j)$ into $(0, L)$, where L is the list of values from these pairs (i.e., the list of partial counts c_j , with $0 \leq j < \sqrt{N}$) and return $(0, \sum_{c_j \in L} c_j)$ as final output.

It is easy to see that the above 4-round algorithm requires local space $M_L = \Theta(\sqrt{N})$ and aggregate space $M_A = \Theta(N)$. As an example, apply the algorithm to the following input

$$S = \{(0, 21), (1, 120), (2, 21), (3, 76), (4, 76), (5, 76), (6, 101), (7, 120), \\ (8, 21), (9, 120), (10, 120), (11, 76), (12, 76), (13, 76), (14, 560), (15, 21)\}.$$

□

Exercise 4 Consider the Class count problem with the input consisting only of object-class pairs (o, γ) , i.e., where no integer keys in $[0, N)$ are provided (now the objects act as keys). Devise an efficient MR algorithm for the problem. (**Hint:** use random keys and target probabilistic guarantees.)

Solution. The algorithm takes inspiration from the Improved Word count algorithm presented in class.

Round 1

- *Map phase*: map each pair (o_i, γ_i) into the intermediate pair $(x, (o_i, \gamma_i))$, where x (the key of the pair) is a random value in $[0, \sqrt{N})$ generated independently and with uniform probability.
- *Reduce phase*: For each key $x \in [0, \sqrt{N})$ separately, gather all intermediate pairs with key x into pair (x, L_x) , where L_x is the list of values (i.e., pairs (o_i, γ_i)) from these pairs. For each distinct class γ occurring in L_x return the pair $(\gamma, c(x, \gamma))$, where $c(x, \gamma)$ is the number of elements $(o_i, \gamma_i) \in L_x$ with $\gamma_i = \gamma$.

Round 2

- *Map phase*: empty
- *Reduce phase*: for each class γ separately, gather all pairs $(\gamma, c(x, \gamma))$ into a pair (γ, L_γ) , where L_γ is the list of values of these pairs (i.e., partial counts $c(x, \gamma)$), and return the pair $(\gamma, c(\gamma) = \sum_{c(x, \gamma) \in L_\gamma} c(x, \gamma))$. Note that $|L_\gamma| \leq \sqrt{N}$.

The analysis is similar to the one presented in class for Improved Word count. Let m_x be the number of intermediate pairs with key x produced by the Map phase of Round 1 (i.e., $m_x = |L_x|$), and let $m = \max_x m_x$. It is easy to see that the above 2-round algorithm requires local space $M_L = O(m + \sqrt{N})$ and aggregate space $M_A = O(N)$. We can prove

that $m = O(\sqrt{N})$ with probability $\geq 1 - 1/N^5$ using exactly the same proof that was used for the theorem in the analysis of Improved Word count. Hence, $M_L = O(\sqrt{N})$ with high probability. $\square$

Exercise 5 *Design and analyze an efficient MR algorithm for approximating the maximum pairwise distance, which uses local space $M_L = O(N^{1/4})$. Can you generalize the algorithm to attain $M_L = O(N^\epsilon)$, for any $\epsilon \in (0, 1/2)$?*

Solution. Recall that the input is a set S of N points represented by pairs (i, x_i) , for $0 \leq i < N$, where x_i is the i -th point. We modify the algorithm seen in class that computes the maximum distance between x_0 and all other points, which is within a factor at most 2 from the actual maximum distance between all points. The input-output specification of the algorithm is the following:

- **Input:** $S = \{(i, x_i) : 0 \leq i < N\}$.
- **Output:** $(0, d_{\max}(0) = \max_{0 \leq i < N} d(x_0, x_i))$.

For simplicity, we assume that x_0 is a global variable and that the distance function is known to the program. The algorithm is the following.

Round 1.

- *Map phase:* map each pair (i, x_i) into the intermediate pair $(i \bmod N^{3/4}, d(x_0, x_i))$.
- *Reduce phase:* for each $j \in [0, N^{3/4})$ separately, gather the intermediate pairs with key j into pair (j, L_j) , where L_j is the list of values (i.e., distances) from these pairs, and return pair $(j, \max\{d \in L_j\})$.

Observation: At this point, $N^{3/4}$ pairs remain, whose keys are the distinct integers in $[0, N^{3/4})$ and whose values are partial maximum distances between x_0 and other points.

Round 2.

- *Map phase:* map each pair (j, d) into the intermediate pair $(j \bmod N^{1/2}, d)$.
- *Reduce phase:* for each $j \in [0, N^{1/2})$ separately, gather the intermediate pairs with key j into pair (j, L_j) , where L_j is the list of values (i.e., distances) from these pairs, and return pair $(j, \max\{d \in L_j\})$.

Observation: At this point, $N^{1/2}$ pairs remain, whose keys are the distinct integers in $[0, N^{1/2})$ and whose values are partial maximum distances between x_0 and other points.

Round 3.

- *Map phase*: map each pair (j, d) into the intermediate pair $(i \bmod N^{1/4}, d)$.
- *Reduce phase*: for each $j \in [0, N^{1/4})$ separately, gather the intermediate pairs with key j into pair (j, L_j) , where L_j is the list of values (i.e., distances) from these pairs, and return pair $(0, \max\{d \in L_j\})$.

Observation: At this point, $N^{1/4}$ pairs remain, all with key 0, whose values are partial maximum distances between x_0 and other points.

Round 4.

- *Map phase*: identity map.
- *Reduce phase*: gather the pairs $(0, d)$ into pair $(0, L)$, where L is the list of values from these pairs, and return $(0, d_{\max}(0) = \max\{d \in L\})$ as final output.

It is easy to see that the above 4-round algorithm uses linear aggregate space. As for the local space, in each round the map phase requires $O(1)$ local space, while the reduce phase, works separately on subsets of size $\Theta(N^{1/4})$ and produces one pair out of each subset. Hence, the local space M_L is $\Theta(N^{1/4})$.

The above algorithm can be easily generalized so to require local space $M_L = O(N^\epsilon)$, for any $\epsilon \in (0, 1/2)$. After all distances between x_0 and the other points are computed, in the map phase of Round 1, then the maximum among these distances is computed in $\Theta(1/\epsilon)$ rounds, where in each round the surviving distances are grouped in subsets of size N^ϵ each and the maximum distance in each subset is returned. $\square$

Exercise 6 Design an $O(1)$ -round MR algorithm for computing a matrix-vector product $W = A \cdot V$, where A is an $m \times n$ matrix, V is an n -vector, and $m \leq \sqrt{n}$. Your algorithm must use $o(n)$ local space and linear, that is, $O(mn)$ aggregate space.

How would your solution change if m were larger?

Solution. The input-output specification of the algorithm is as follows. We assume that A is represented through the mn pairs $((i, j), A[i, j])$, with $0 \leq i < m$ and $0 \leq j < n$, where (i, j) is the key, and that V is represented through the n pairs $((j, -1), V[j])$, with $0 \leq j < n$, where $(j, -1)$ is the key. In this fashion we use the same representation for entries of A and V , and, in particular, we use keys from the same domain for both. As for W , we assume that in output it will be represented by the pairs $(i, W[i])$, with $0 \leq i < m$.

Note that $W[i]$ is the inner product of a row of A and V , which are both vectors of size n . A straightforward 1-round approach would be to make m copies of V in the map phase, and then, in the reduce phase, to compute $W[i]$, separately for $i \in [0, m)$, by gathering row i of A and copy i of V , and computing their inner product. However, this approach requires $\Theta(n)$ local space. In order to get $o(n)$ local space, we can break V and each row of A into $\sqrt{n}$ segments of size $\sqrt{n}$ each. In the first round, we compute separately the product of each segment of each row i of A and the corresponding segment of V , which yields $\sqrt{n}$ contributions to every $W[i]$. In the second round we just add up these $\sqrt{n}$ contributions, separately for every $W[i]$. The following algorithm implements this idea.

Round 1

- *Map phase*: map each pair $((i, j), A[i, j])$ into the intermediate pair $((i, s = \lfloor j/\sqrt{n} \rfloor), (i, j, A[i, j]))$, and each pair $((j, -1), V[j])$ into the set of intermediate pairs $\{(i, s = \lfloor j/\sqrt{n} \rfloor), (j, -1, V[j])\} : 0 \leq i < m\}$.

Notation: we let $A^{(i,s)}$ denote segment s of row $A^{(i)}$, and let $V^{(i,s)}$ denote segment s of the i -th copy of V . Observe that both $A^{(i,s)}$ and $V^{(i,s)}$ contain (at most) $\sqrt{n}$ entries.

- *Reduce phase*: for each key (i, s) separately, with $0 \leq i < m$ and $0 \leq s < \sqrt{n}$, gather all pairs with key (i, s) into a pair $((i, s), L_{i,s})$, where $L_{i,s}$ is the list of values from these pairs (i.e., $L_{i,s}$ contains all entries of $A^{(i,s)}$ and $V^{(i,s)}$), and return the pair $(i, W_s[i] = A^{(i,s)} \circ V^{(i,s)})$. (The operator " $\circ$ " denotes the inner product.)

Round 2

- *Map phase*: empty
- *Reduce phase*: for each key i separately, gather all pairs $(i, W_s[i])$, with $0 \leq s < \sqrt{n}$, into a pair (i, L_i) , where L_i is the list all $W_s[i]$'s, and return the output pair $(i, W[i] = \sum_{W_s[i] \in L_i} W_s[i])$.

For what concerns the local space, the map phase of Round 1 works on a single key value pair and transforms such a pair into 1 or $m \leq \sqrt{n}$ intermediate pairs, depending on whether it comes from A or V . The reduce phases of both rounds process sets of $O(\sqrt{n})$ entries each. Thus, we conclude that $M_L \in O(\sqrt{n})$. As for the aggregate space, A is not replicated, while the m replicas of V occupy overall $O(mn)$ space, that is, the same space as A . Thus, $M_A \in O(mn)$, that is linear in the input size.

If m were larger, the replication of each entry of V into m copies performed in Round 1 would now be accomplished in several rounds, where in each round every current copy is replicated $\sqrt{n}$ times. For example, if $\sqrt{n} < m \leq n$, two rounds would suffice. The details are left as an exercise. $\square$